

apr 27, 2025 9:23 k-diffusion-pinn-5200-half-channel-plus-units-load-skip-5-cells.py Page 3/6

```

vist_1 = vist_DNS[-1]

y = yplus_DNS

# Define get_derivative
dtype = torch.float
device = torch.device("cpu")

def get_derivative(f, y):
    """Compute the derivative of f=f(y) with respect to y."""
    df_dy = grad(f, y, torch.ones(y.size()[0], 1, device=device), create_graph=True)[0]
    return df_dy

class MyNet2(nn.Module):
    def __init__(self):
        super().__init__()
        self.ll1 = nn.Linear(in_features=1, out_features=10)
        self.tanh = nn.Tanh()
        self.ll2 = nn.Linear(in_features=10, out_features=10)
        self.ll3 = nn.Linear(in_features=10, out_features=10)
        self.output = nn.Linear(in_features=10, out_features=1)

    def forward(self, x):
        out = self.ll1(x)
        out = self.tanh(out)
        out = self.ll2(out)
        out = self.tanh(out)
        out = self.ll3(out)
        out = self.output(out)
        return out

# Create an instance
model = MyNet2()

%% Define loss function
def PDE(y, vist_pred):
    """Compute the cost function."""
    global temp
    # Differential equation loss
    dvist_dy = get_derivative(vist_pred, y)
    temp = (vist_pred + viscos_lam) * d2kdy2_DNS + dkdy_DNS * dvist_dy

    boundary_condition_loss = 0
    differential_equation_loss = temp + (Pk_DNS - diss_DNS)
    imbalance = differential_equation_loss
    differential_equation_loss = torch.sum(differential_equation_loss ** 2)
    # Boundary condition loss initialization
    boundary_condition_loss = 0
    # Sum over dirichlet boundary condition losses
    boundary_condition_loss += (vist_pred[0] - vist_0) ** 2
    boundary_condition_loss += (vist_pred[-1] - vist_1) ** 2

    return differential_equation_loss, boundary_condition_loss, imbalance

def loss_and_PDE(y_tensor):
    optimizer.zero_grad() # Clear gradients from the previous iteration
    outputs = model(y_tensor) # get vist_t
    loss_de, loss_bc, imbalance = PDE(y_tensor, outputs) # Compute the loss
    loss = loss_de + 1000. * loss_bc
    # Calculate the L1 regularization term
    ll_regularization = torch.tensor(0.)

```

apr 27, 2025 9:23 k-diffusion-pinn-5200-half-channel-plus-units-load-skip-5-cells.py Page 4/6

```

for param in model.parameters():
    ll_regularization += torch.norm(param, p=1)

# Add the L1 regularization term to the loss
lambda_ll = 0.
loss += lambda_ll * ll_regularization # Compute the loss
loss.backward() # Compute gradients using backpropagation
return loss, loss_de, loss_bc, imbalance

%% training
prev_loss = float('inf') # Initialize with a large value
max_no_epoch = 100000
# max_no_epoch = 2

optim_alg = 'Adam'
# optim_alg = 'SGD'
learning_rate = 0.2 # 4221.8496 milestones=[500000]
learning_rate = 0.2 # 0.6554632 milestones=[6400, 43000, 54578]
print('Adam taken')
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# for saving training result
differential_equation_loss_history = np.zeros(max_no_epoch)
boundary_condition_loss_history = np.zeros(max_no_epoch)
loss_min = 1e30
# Training loop
for epoch in range(max_no_epoch):
    loss, loss_de, loss_bc, imbalance = loss_and_PDE(y)
    differential_equation_loss_history[epoch] += loss_de
    boundary_condition_loss_history[epoch] += loss_bc
    optimizer.step()
    loss_change = prev_loss - loss
    prev_loss = loss

# Define checkpoint
if epoch == 0:
    checkpoint = torch.load('checkpoint-vist-5200-plus-units-save-5-cells.ct')

# Apply the state_dict to model and optimizer
model = MyNet2() # Initialize model; Ensure it's the same architecture
model.load_state_dict(checkpoint['model_state_dict'])

optimizer = optim.Adam(model.parameters(), lr=learning_rate) # Initialize
optimizer; Ensure it's the same optimizer type
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[6400, 16700, 19200, 37500, 38000, 80000, 94000], gamma=0.5)

# Retrieve the training epoch
epoch = checkpoint['epoch']
loss = checkpoint['loss']

model.train() # For training mode (resuming training)

scheduler.step()

loss_np = loss.detach().numpy()

# Print the loss every epoch
loss_min = np.minimum(loss_np, loss_min)
loss_min = np.float32(loss_min)
torch.set_printoptions(precision=4)

```